# Linux & Python
# Daily Spiral Learning Plan

A cumulative, repetition-based path from zero to DevOps-ready basics

---

**How this plan works**

Each day starts by **repeating everything from every previous day** before adding anything new. You are not expected to remember it perfectly the first time — the repetition is the point. New material is added in small doses (1–3 concepts per day) on top of the review, so by the time you reach Day 10 you've typed Day 1's commands ten separate times without ever sitting down to "memorize" them.

Two tracks are included: **Linux** (14 days) and **Python** (14 days). Run them in parallel — one Linux session and one Python session per day — or back to back if you'd rather finish one before starting the other. Each session should take 20–40 minutes.

Every day ends with homework: a small task to complete before the next session, which becomes part of the next day's review.

# LINUX TRACK — 14 DAYS

## DAY 1: Who am I, where am I, moving around

### ■ NEW TODAY

**whoami** — prints the username you're logged in as

```
whoami
```

**pwd** — prints your current directory ("print working directory")

```
pwd
```

**cd** — changes directory: into a folder, up one level, home, and root

```
cd Documents
cd ..
cd ~
cd /
```

**ls** — lists what's in the current directory

```
ls
ls -l
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Run whoami, then pwd, five times in a row without looking at your notes.

• cd into a folder, run pwd to confirm, then cd .. back out.

• cd ~ to jump home from anywhere, then confirm with pwd.

### ■ HOMEWORK (do before tomorrow)

■ Open a terminal from a cold start (no notes) and, from memory, run whoami, pwd, ls, cd into any folder, pwd again, cd back home.

■ Write down (on paper or in a notes app) what pwd, cd, and ls each do, in your own words.

## DAY 2: Making things: directories and files

### ■ REVIEW — do these first, from memory before checking notes

• whoami, pwd

• cd into a folder / cd .. / cd ~

• ls and ls -l

### ■ NEW TODAY

**mkdir** — creates a new directory

```
mkdir practice
mkdir practice/subfolder
```

**touch** — creates an empty file

```
touch notes.txt
```

**nano (editing + saving)** — opens a simple text editor; type text, then Ctrl+O to save, Enter to confirm, Ctrl+X to exit

```
nano notes.txt
```

**chmod (intro)** — makes a file executable; full permission system comes on Day 8

```
chmod +x script.sh
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• mkdir a new folder, cd into it, pwd to confirm you're inside it.

• touch three different files in a row, then ls to see them all.

• Open one file with nano, type a sentence, save, exit, then cat it (preview of Day 4) to see it worked — just look, don't worry about cat yet.

### ■ HOMEWORK (do before tomorrow)

■ Create a folder called linux_practice in your home directory with three files inside it, using only the terminal.

■ Repeat Day 1's full sequence from memory before doing today's homework.

## DAY 3: Copy, move, delete, and listing details

### ■ REVIEW — do these first, from memory before checking notes

• whoami, pwd, cd, ls

• mkdir, touch, nano save/exit

### ■ NEW TODAY

**ls -la** — shows hidden files and permission details — you'll read this fully on Day 8

```
ls -la
```

**cp** — copies a file, or a whole folder with -r

```
cp notes.txt notes_backup.txt
cp -r practice practice_copy
```

**mv** — moves or renames a file

```
mv notes_backup.txt archive.txt
mv archive.txt practice/
```

**rm** — deletes a file, or a folder with -r — there is no undo, be careful

```
rm archive.txt
rm -r practice_copy
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Create a file, copy it, rename the copy with mv, then delete the original.

• Create a folder with a file in it, copy the whole folder with cp -r, then remove the copy with rm -r.

• Run ls -la in your home folder and try to identify which entries are files vs. folders (folders start with 'd').

### ■ HOMEWORK (do before tomorrow)

■ In linux_practice, create a file, make a backup copy of it, move the backup into a new subfolder, then delete the subfolder entirely.

■ From memory, repeat Days 1–2's full command list once before starting.

# DAY 4: Looking inside files

## ■ REVIEW — do these first, from memory before checking notes

• whoami, pwd, cd, ls, ls -la

• mkdir, touch, nano

• cp, mv, rm

## ■ NEW TODAY

**cat** — dumps a whole file's contents to the screen

```
cat notes.txt
```

**less** — opens a file for scrollable reading — q to quit

```
less notes.txt
```

**head / tail** — shows the first/last lines of a file; -f follows a growing log live

```
head notes.txt
tail notes.txt
tail -f logfile.txt
```

## ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Write a few lines into a file with nano, then view it three different ways: cat, less, head.

• Practice quitting less with q at least three times so it becomes automatic.

• cat two different files back to back to compare their contents.

## ■ HOMEWORK (do before tomorrow)

■ Create a file with at least 15 lines of text (copy/paste is fine) and practice head, tail, and less on it.

■ Redo Day 1's and Day 3's practice steps from memory, back to back.

# DAY 5: Redirecting output and piping commands together

## ■ REVIEW — do these first, from memory before checking notes

• Full file navigation (pwd/cd/ls)

• File creation & editing (mkdir/touch/nano)

• Copy/move/delete (cp/mv/rm)

• Viewing files (cat/less/head/tail)

## ■ NEW TODAY

**> and >>** — > overwrites a file with output, >> appends to it

```
ls -la > filelist.txt
echo "hello" >> notes.txt
```

**| (pipe)** — sends one command's output into another command as input

```
ls -la | less
cat notes.txt | head
```

**echo** — prints text — useful for testing and for writing into files

```
echo "Hello World"
```

## ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Redirect the output of ls -la into a new file, then cat that file to confirm it worked.

• Use echo with >> three separate times to build up a file line by line.

• Pipe ls -la into less so you can scroll a long directory listing.

## ■ HOMEWORK (do before tomorrow)

■ Create a file called system_info.txt that contains the output of whoami, pwd, and ls -la, each appended in turn using >>.

■ Repeat Day 2's and Day 4's practice from memory before starting.

# DAY 6: Searching inside files with grep

## ■ REVIEW — do these first, from memory before checking notes

• Navigation + files (Days 1–4 commands)

• Redirects > >>

• Pipe |

## ■ NEW TODAY

**grep** — searches for text inside files; -i ignores case, -r searches recursively through folders

```
grep "hello" notes.txt
grep -i "hello" notes.txt
grep -r "TODO" .
cat notes.txt | grep "hello"
```

## ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Search a file for a word you know is in it, then a word you know is NOT in it — notice grep prints nothing when there's no match.

• Try grep -i to confirm case doesn't matter.

• Combine grep with a pipe: cat a file and grep it for a word in one line.

## ■ HOMEWORK (do before tomorrow)

■ In linux_practice, create two files with different sentences, then use grep -r to search both files at once for a common word.

■ Redo Day 5's homework from memory (rebuild system_info.txt without looking).

# DAY 7: Finding files anywhere on the system

## ■ REVIEW — do these first, from memory before checking notes

• cat/less/head/tail

• > >> and |

• grep and grep -i / -r

## ■ NEW TODAY

**find** — searches for files/folders by name or type, starting from a given path

```
find . -name "notes.txt"
find . -type d
find / -name "*.log" 2>/dev/null
```

**wildcard * (intro)** — * matches "anything" in a filename pattern

```
ls *.txt
find . -name "*.txt"
```

## ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Use find . -name to locate a file you know exists in your practice folder.

• Use find . -type d to list only directories inside a folder.

• Use ls *.txt to list only .txt files in the current folder.

### ■ HOMEWORK (do before tomorrow)

■ Create three .txt files and one .log file in a folder, then use find to list only the .txt files.

■ Repeat Day 3 and Day 6's practice from memory.

## DAY 8: Permissions and ownership, properly

### ■ REVIEW — do these first, from memory before checking notes

• find and wildcards

• grep basics

• Redirects and pipes

### ■ NEW TODAY

**Reading ls -la permissions** — format is type+permissions (e.g. -rwxr-xr--), then owner, group, size, date, name

```
ls -la
```

**chmod (numeric)** — sets permissions numerically: read=4, write=2, execute=1, added per owner/group/others

```
chmod 644 notes.txt
chmod 755 script.sh
```

**chown** — changes who owns a file (needs sudo on most systems)

```
sudo chown $USER file.txt
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Run ls -la and pick three files; for each, say out loud what the permission letters mean.

• chmod a file to 644, then to 755, and run ls -la after each to see the difference.

• Try running a script you haven't made executable (it fails), then chmod +x it and run it again.

### ■ HOMEWORK (do before tomorrow)

■ Create a script file (even one line: echo hello), make it non-executable on purpose, try to run it, then fix it with chmod and run it successfully.

■ Redo Day 1's full sequence and Day 7's find practice from memory.

## DAY 9: Seeing and controlling running processes

### ■ REVIEW — do these first, from memory before checking notes

• chmod numeric permissions

• ls -la reading

• find + grep

### ■ NEW TODAY

**ps** — lists running processes; aux shows every process on the system

```
ps
ps aux
```

**top / htop** — live, updating view of processes and resource usage — q to quit

```
top
htop
```

**kill** — stops a process by its process ID (PID); -9 forces it

```
kill 1234
kill -9 1234
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Run ps aux and find your own shell process in the list.

• Open top, watch it for 30 seconds, then quit with q.

• Start a harmless long-running command (e.g. 'sleep 100'), find its PID with ps, then kill it.

### ■ HOMEWORK (do before tomorrow)

■ Start a sleep 300 command in one terminal tab (or with & to background it), find it with ps aux, and kill it before it finishes on its own.

■ Repeat Day 5's redirect homework and Day 8's chmod exercise from memory.

## DAY 10: Installing and managing software

### ■ REVIEW — do these first, from memory before checking notes

• ps / top / kill

• Permissions (chmod/chown)

• grep + find

### ■ NEW TODAY

**apt update / upgrade** — refreshes the list of available packages, then upgrades installed ones

```
sudo apt update
sudo apt upgrade
```

**apt install / remove** — installs or removes a package

```
sudo apt install tree
sudo apt remove tree
```

**apt search** — searches for available packages by keyword

```
apt search text editor
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Run sudo apt update and read the output — you don't need to understand every line, just get comfortable seeing it run.

• Install a small, harmless tool (like tree) and run it.

• Remove that same tool afterward with apt remove.

### ■ HOMEWORK (do before tomorrow)

■ Install the 'tree' package, use it to view your linux_practice folder structure, then uninstall it.

■ Redo Day 9's process-killing exercise and Day 6's grep homework from memory.

# DAY 11: Environment variables and PATH

## ■ REVIEW — do these first, from memory before checking notes

• apt update/install/remove

• ps/top/kill

• Permissions

## ■ NEW TODAY

**echo $VAR** — prints the value of an environment variable

```
echo $HOME
echo $PATH
echo $USER
```

**export** — creates a new environment variable for the current session

```
export MY_NAME="Alex"
echo $MY_NAME
```

**Understanding PATH** — PATH is the list of folders the shell searches for commands; which shows where a command lives

```
echo $PATH
which python3
```

## ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Print $HOME, $PATH, and $USER, and describe in your own words what each one is.

• export a variable with your own name, then echo it back.

• Run which on three different commands you've used this week (ls, grep, nano) to see where they live.

## ■ HOMEWORK (do before tomorrow)

■ Export a variable called PROJECT_DIR set to the path of your linux_practice folder, then cd $PROJECT_DIR to prove it works.

■ Repeat Day 10's install/remove cycle and Day 4's viewing commands from memory.

# DAY 12: Your first shell scripts

## ■ REVIEW — do these first, from memory before checking notes

• Environment variables, export, PATH

• apt basics

• Permissions

## ■ NEW TODAY

**Script structure** — the #! line ("shebang") tells the system which interpreter to use

```
#!/bin/bash
echo "Hello from a script"
```

**Writing + running a script** — write it, make it executable, run it with ./

```
nano greet.sh
chmod +x greet.sh
```

```
./greet.sh
```

**Variables in a script** — bash variables — no spaces around the =

```
name="Alex"
echo "Hello, $name"
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Write a script that echoes your name and today's date-like message, then run it three separate times, editing it slightly each time.

• Add a variable to your script and reference it with $ inside an echo.

• Practice the full cycle from empty file to running script five times: nano, save, chmod +x, ./

### ■ HOMEWORK (do before tomorrow)

■ Write a script called intro.sh that prints your name, your current directory (using pwd inside the script), and a farewell message.

■ Redo Day 11's export/PATH exercise and Day 2's file-creation basics from memory.

## DAY 13: Conditionals, loops, and arguments in scripts

### ■ REVIEW — do these first, from memory before checking notes

• Writing & running a script (shebang, chmod +x, ./)

• Variables in bash

• Environment variables

### ■ NEW TODAY

**if / else** — bash conditional — note the spaces inside [ ] are required

```
if [ "$name" == "Alex" ]; then
echo "Hi Alex"
else
echo "Who are you?"
fi
```

**for loop** — repeats a block of commands for each item in a list

```
for i in 1 2 3; do
echo "Number $i"
done
```

**Script arguments** — $1 is the first argument passed to the script when you run it

```
./greet.sh Alex
echo "Hello, $1"
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Add an if/else to a script that checks a variable and prints different things depending on its value.

• Write a for loop that echoes numbers 1 through 5, then modify it to loop over words instead of numbers.

• Modify a script to accept your name as an argument ($1) instead of hardcoding it, and run it with different names.

### ■ HOMEWORK (do before tomorrow)

■ Write a script called checker.sh that takes a name as an argument, and uses if/else to print a personalized greeting if the name matches yours, or a generic one otherwise.

■ Repeat Day 12's script-writing cycle and Day 9's ps/kill exercise from memory.

# DAY 14: Scheduling tasks + full review capstone

## ■ REVIEW — do these first, from memory before checking notes

• if/else and for loops in bash

• Script arguments ($1)

• Environment variables and PATH

• Permissions and processes

## ■ NEW TODAY

**cron / crontab** — schedules a command to run automatically on a recurring basis; format is minute hour day month weekday

```
crontab -e
*/5 * * * * echo "hi" >> /home/you/cronlog.txt
```

**crontab -l** — lists your currently scheduled cron jobs

```
crontab -l
```

## ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Schedule a cron job that appends a line to a log file every minute, then check the file after a few minutes to confirm it ran.

• Remove the cron job afterward (crontab -e and delete the line, or crontab -r to clear all).

• CAPSTONE: starting from a completely blank terminal and no notes, in order: whoami, pwd, mkdir a folder, cd into it, create a script with nano that uses a variable and a for loop, chmod it, and run it.

## ■ HOMEWORK (do before tomorrow)

■ Write one paragraph, in your own words, summarizing what pwd/cd/ls, mkdir/touch/nano, cp/mv/rm, cat/less/grep/find, chmod, ps/kill, and a basic bash script are each for.

■ Keep linux_practice around — you'll use it again when you start Docker and Ansible later.

# PYTHON TRACK — 14 DAYS

## DAY 1: Running Python and printing things

### ■ NEW TODAY

**Running a script** — how you run any Python file from the terminal

```
python3 hello.py
```

**print()** — outputs text or values to the screen

```
print("Hello, world")
print(1 + 1)
```

**Comments** — notes to yourself in the code

```
# this is a comment, Python ignores it
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Write a file hello.py with three different print() statements, and run it from the terminal.

• Add a comment above each print() explaining what it does.

• Change the text inside print() and re-run the script five times.

### ■ HOMEWORK (do before tomorrow)

■ Write a script called about_me.py with at least 4 print() statements: your name, your goal, today's plan, and one fact about yourself.

■ Practice running it with python3 about_me.py from a completely closed terminal.

## DAY 2: Variables and basic data types

### ■ REVIEW — do these first, from memory before checking notes

• print() and running a script with python3

### ■ NEW TODAY

**Variables** — a variable stores a value under a name — no type declaration needed

```
name = "Alex"
age = 30
height = 5.9
```

**Data types** — str (text), int (whole number), float (decimal number)

```
type(name)
type(age)
type(height)
```

**f-strings** — the cleanest way to insert variables into printed text

```
print(f"My name is {name}")
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Create a variable for your name, age, and one number with a decimal, then print all three using an f-string.

• Use type() on each variable to confirm str, int, and float.
• Change a variable's value and re-run the print to see it update.

### ■ HOMEWORK (do before tomorrow)

■ Update about_me.py to use variables and f-strings instead of hardcoded text in every print().
■ Redo Day 1's practice from memory before starting.

## DAY 3: Getting input and making decisions

### ■ REVIEW — do these first, from memory before checking notes

• Variables (str/int/float)
• print() and f-strings

### ■ NEW TODAY

**input()** — pauses the script and waits for the user to type something; always returns a string

```
name = input("What's your name? ")
```

**if / elif / else** — Python uses indentation (not brackets) to define blocks

```
if age >= 18:
print("Adult")
elif age >= 13:
print("Teen")
else:
print("Kid")
```

**Comparison operators** — used inside if statements to compare values

```
==
!=
>
<
>=
<=
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Write a script that asks for your name with input() and greets you with an f-string.
• Write an if/elif/else that checks a number variable against a few ranges and prints different messages.
• Deliberately break the indentation once to see the error Python gives you, then fix it.

### ■ HOMEWORK (do before tomorrow)

■ Write a script called age_check.py that asks the user's age with input() (remember to convert it with int()) and prints whether they're a kid, teen, or adult.
■ Redo Day 2's variables/f-strings homework from memory.

## DAY 4: Lists — storing many things

### ■ REVIEW — do these first, from memory before checking notes

• input() and int() conversion
• if/elif/else
• Comparison operators

### ■ NEW TODAY

**Creating a list** — an ordered collection of values

```
fruits = ["apple", "banana", "cherry"]
```

**Indexing** — 0 is the first item, -1 is the last

```
fruits[0]
fruits[-1]
```

**Common list methods** — add, remove, and count items

```
fruits.append("mango")
fruits.remove("banana")
len(fruits)
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Create a list of 5 items and print the first, last, and third item using indexing.
• Append two new items to the list, then remove one, printing the list after each change.
• Print len() of the list before and after your changes.

### ■ HOMEWORK (do before tomorrow)

■ Write a script called shopping_list.py that builds a list of at least 5 items using input() in a loop-free way (just five separate input() calls appended to a list), then prints the full list and its length.

■ Redo Day 3's age_check.py from memory.

## DAY 5: For loops

### ■ REVIEW — do these first, from memory before checking notes

• Lists: create, index, append, remove, len
• if/elif/else

### ■ NEW TODAY

**for loop over a list** — runs the indented block once per item in the list

```
for fruit in fruits:
print(fruit)
```

**range()** — generates a sequence of numbers, 0 up to (not including) 5

```
for i in range(5):
print(i)
```

**Combining loop + if** — loops and conditionals nest together constantly

```
for fruit in fruits:
if fruit == "banana":
print("Found it")
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Loop over your shopping list and print each item with a number in front of it (hint: use enumerate() if you want to try it, or just a counter variable).

• Use range(10) to print numbers 0 through 9.

• Loop over a list and use an if inside the loop to print only items matching some condition.

## ■ HOMEWORK (do before tomorrow)

■ Rewrite shopping_list.py so a for loop prints each item on its own line prefixed with '- '.

■ Redo Day 4's list practice from memory.

# DAY 6: While loops

## ■ REVIEW — do these first, from memory before checking notes

• for loops and range()

• Lists

## ■ NEW TODAY

**while loop** — repeats as long as a condition is True — must update the condition inside the loop or it never ends

```
count = 0
while count < 5:
print(count)
count += 1
```

**break and continue** — break exits the loop entirely, continue skips to the next iteration

```
if count == 3:
break
if count == 2:
continue
```

## ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Write a while loop that counts from 0 to 10.

• Add a break so the loop stops early at a number of your choice.

• Write a while loop that keeps asking the user for input() until they type 'quit'.

## ■ HOMEWORK (do before tomorrow)

■ Write a script called countdown.py that counts down from 10 to 1 using a while loop, then prints 'Liftoff!'.

■ Redo Day 5's for-loop homework and Day 3's if/elif/else from memory.

# DAY 7: Functions

## ■ REVIEW — do these first, from memory before checking notes

• while loops, break/continue

• for loops

## ■ NEW TODAY

**Defining a function** — def starts a function definition; name is a parameter

```
def greet(name):
print(f"Hello, {name}")
```

**Calling a function** — runs the function with a specific value

```
greet("Alex")
```

**return** — sends a value back out of the function to be used elsewhere

```
def add(a, b):
return a + b
result = add(2, 3)
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Write a function that takes a name and returns a greeting string instead of printing it directly, then print() the result.

• Write a function that takes two numbers and returns their sum, then call it three times with different numbers.

• Turn one of your earlier scripts' logic (like age_check) into a function you can call with different ages.

### ■ HOMEWORK (do before tomorrow)

■ Write a script called calculator.py with functions add(a,b), subtract(a,b), multiply(a,b), and divide(a,b), and call each one with sample numbers, printing the results.

■ Redo Day 6's countdown.py and Day 4's list-loop from memory.

## DAY 8: Dictionaries

### ■ REVIEW — do these first, from memory before checking notes

• Functions: def, parameters, return

• for/while loops

### ■ NEW TODAY

**Creating a dictionary** — stores key-value pairs instead of a plain ordered list

```
person = {"name": "Alex", "age": 30}
```

**Accessing values** — look up a value by its key

```
person["name"]
person.get("age")
```

**Looping over a dictionary** — iterates over both keys and values at once

```
for key, value in person.items():
print(key, value)
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Create a dictionary describing yourself (name, age, goal) and print each value by key.

• Add a new key-value pair to the dictionary, then update an existing value.

• Loop over the dictionary with .items() and print 'key: value' for each pair.

### ■ HOMEWORK (do before tomorrow)

■ Write a script called contact.py that stores a dictionary for a contact (name, phone, email) and prints a formatted 'contact card' using an f-string and the dictionary values.

■ Redo Day 7's calculator.py from memory.

## DAY 9: Reading and writing files

### ■ REVIEW — do these first, from memory before checking notes

• Dictionaries: create, access, loop with .items()

• Functions

### ■ NEW TODAY

**Writing a file** — "w" creates/overwrites the file; the with block auto-closes it

```
with open("data.txt", "w") as f:
f.write("Hello file\n")
```

**Appending to a file** — "a" adds to the end instead of overwriting

```
with open("data.txt", "a") as f:
f.write("Another line\n")
```

**Reading a file** — "r" reads; you can also loop line by line with 'for line in f:'

```
with open("data.txt", "r") as f:
contents = f.read()
print(contents)
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Write a script that writes three lines to a file using 'w' mode, then a separate run that appends a fourth line using 'a'.

• Read the whole file back with .read() and print it.

• Read the file line by line with a for loop over the file object instead of .read().

### ■ HOMEWORK (do before tomorrow)

■ Write a script called journal.py that asks the user with input() for one sentence and appends it, with today's placeholder label, to a file called journal.txt each time it's run.

■ Redo Day 8's contact.py and Day 6's function practice from memory.

## DAY 10: Handling errors gracefully

### ■ REVIEW — do these first, from memory before checking notes

• File reading/writing (open, with, 'r'/'w'/'a')

• Dictionaries

### ■ NEW TODAY

**try / except** — try runs code that might fail; except catches the specific error instead of crashing

```
try:
age = int(input("Age: "))
except ValueError:
print("That's not a number")
```

**Catching file errors** — same pattern applied to files that might not exist

```
try:
open("missing.txt")
except FileNotFoundError:
print("File not found")
```

**finally** — runs no matter what, whether it succeeded or failed

```
finally:
print("Done trying")
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Wrap an int(input()) conversion in try/except so a non-number input doesn't crash the script.

• Try to open a file that doesn't exist and catch the FileNotFoundError with a friendly message.
• Add a finally block that always prints something after the try/except runs.

## ■ HOMEWORK (do before tomorrow)

■ Update age_check.py from Day 3 to use try/except so it doesn't crash when the user types letters instead of a number.

■ Redo Day 9's journal.py and Day 7's dictionary practice from memory.

# DAY 11: Modules and the os module

## ■ REVIEW — do these first, from memory before checking notes

• try/except/finally

• Functions

• File reading/writing

## ■ NEW TODAY

**import** — brings in extra functionality from Python's standard library

```
import os
import math
```

**os basics** — Python equivalents of pwd, ls, and checking if a file exists

```
os.getcwd()
os.listdir(".")
os.path.exists("data.txt")
```

**Your own modules** — any .py file you write can be imported into another script

```
import calculator
calculator.add(2,3)
```

## ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Import os and print the current working directory and a listing of files in it.

• Use os.path.exists() to check for a file you know exists, and one you know doesn't.

• Import your calculator.py from Day 7 into a new script and call one of its functions.

## ■ HOMEWORK (do before tomorrow)

■ Write a script called file_check.py that uses os.path.exists() to check for journal.txt, and prints a different message depending on whether it's there.

■ Redo Day 10's try/except homework and Day 8's dictionary loop from memory.

# DAY 12: Running shell commands from Python (subprocess)

## ■ REVIEW — do these first, from memory before checking notes

• import and the os module

• try/except

## ■ NEW TODAY

**subprocess.run** — runs a shell command from inside a Python script — this is the bridge between your two tracks

```
import subprocess
subprocess.run(["ls", "-la"])
```

**Capturing output** — captures the command's output as a string instead of just printing it to the terminal

```
result = subprocess.run(["whoami"], capture_output=True, text=True)
print(result.stdout)
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Use subprocess.run to call whoami and pwd from inside a Python script.

• Capture the output of ls -la with capture_output=True and print result.stdout.

• Try running a command that doesn't exist and see what happens in result.stderr (add text=True).

### ■ HOMEWORK (do before tomorrow)

■ Write a script called sysinfo.py that uses subprocess to run whoami, pwd, and ls -la, captures each output, and writes all three into a file called sysinfo_report.txt.

■ Redo Day 11's file_check.py and Day 9's try/except practice from memory.

## DAY 13: Introduction to classes

### ■ REVIEW — do these first, from memory before checking notes

• subprocess.run and capture_output

• Functions

• Dictionaries

### ■ NEW TODAY

**Defining a class** — a class is a blueprint; __init__ sets up a new object's starting data

```
class Dog:
def __init__(self, name):
self.name = name
def bark(self):
print(f"{self.name} says woof")
```

**Creating an object** — an 'instance' of the class, with its own data

```
my_dog = Dog("Rex")
my_dog.bark()
```

### ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• Write a simple class (e.g. Person with a name and a greet() method), create two different instances, and call greet() on each.

• Add a second attribute to the class (like age) and print it from an instance.

• Add a second method to the class that uses self to reference the object's own data.

### ■ HOMEWORK (do before tomorrow)

■ Write a script called contact_class.py that turns Day 8's contact dictionary into a Contact class with name/phone/email attributes and a method that prints a formatted card.

■ Redo Day 12's sysinfo.py and Day 10's os practice from memory.

## DAY 14: Capstone: a script that automates a Linux task

### ■ REVIEW — do these first, from memory before checking notes

• Classes

• subprocess

• Functions, loops, try/except, file I/O — everything so far

## ■ NEW TODAY

**Putting it together** — today's goal is composition, not new material

```
# no new syntax today — just combining what you know
```

## ■ PRACTICE — repeat each of these 3–5 times until it feels automatic

• CAPSTONE, no notes: write a script that (1) uses subprocess to list files in a folder, (2) loops over the result, (3) uses try/except to handle any errors, (4) writes a summary to a file using 'with open'.

• Add a function that wraps the whole thing so it can be called with a different folder path as an argument.

• Add a small class that stores the folder path and the summary as attributes, if you want an extra challenge.

## ■ HOMEWORK (do before tomorrow)

■ Finish and save your capstone script as folder_report.py. This is your first real 'automation script' — the kind of thing you'll build constantly in DevOps work.

■ Write one paragraph, in your own words, listing every Python concept from these 14 days you used in the capstone.

# What's next

By the end of these 28 days you'll have typed the Linux fundamentals and Python fundamentals dozens of times each, and built a small library of your own scripts. From here, the natural next steps toward DevOps are: Git/GitHub for version control, Docker for containers, a CI/CD tool like GitHub Actions, Ansible for configuration management (it's Python-based, so it will feel familiar), and a lightweight Kubernetes distribution like k3s or minikube for orchestration. Keep using this same spiral method — review everything old before adding anything new — and it will keep working just as well for those tools.